

```
clear
clc
clf
%% Approximating ODEs
%{
You dug a tunnel straight through the center of the earth to jump
* g - acceleration/deceleration due to gravity which changes
  w/respect to the radius of the earth & a dragforce that is
  proportional to the observers velocity in the opposing direction
  of transit.
```

Governing equation of motion expressed as ODE for velocity:

$$m \frac{dv}{dt} = m \cdot \text{sign}(r) g(r) - \text{sign}(v) \cdot C_d \cdot v^2$$

Where:

```
v = velocity
t = time(s)
g(r) - the changing gravitational constant (m/s2) as a
       function of height above the Earths center (r, in m)
sign(x) - function that returns -1, 0, and 1 for:
          negative, zero, and positive values of x.
C_d is the drag coefficient (kg/m)
m = mass (kg)
```

Acceleration due to gravity:

$$g(r) = (4\pi/3) \cdot G \cdot \rho \cdot r$$

Where:

```
G = 6.674 × 10(-11) m(3) / kg*s(2) - Gravitaional Constant
ρ = 5515 kg/m(3) - average density of Earth
```

Calculate the Postion and Velocity of the jumper for:

```
t = 0 - 3*10(5) s (approx. 80 hours)
```

Given:

```
Constants: m= 65.8 kg , C_d = 0.25 kg/m
```

```
Initial Conditions: r = 6.371*10(6) m , v_o = 0 m/s
```

```
%}
```

```
% time span
```

```
tspan=linspace(0,3*10(5));
```

```
% constants
```

```

G = 6.674*10^(-11); % m^(3) / kg*s^(2) - Gravitaional Constant
rho = 5515; % kg/m^(3) - average density of Earth
m = 65.8; % kg
C_d = 0.25; % kg/m

%initial Conditions r_0 =[r0 , r'0]

r_0 = [6.371e6, 0];
%%
%% Setting up the lineariztation of the 2nd order non-linear ODE
%{
because v = velocity & r is the displacement of the falling object v' is
acceleration. We can relate our function to as the general form equation:

    Displacement r(t) = r + v*t + 0.5*a*t^(2)

which is in the form of a taylor series where (v = r') & (a = v' = r'')
therefore we can replace the v and v' in our orignal governing equation.

    m*r'' = m*sign(r)*g(r) - sign(r')*C_d*r'^2
    generalized to y'' + a*y' - b*y = 0
    ~ r'' + a*r' - b*r = 0
    therefor we can turn in system of 1st order ODEs

r1 = -m*sign(r)*g(r) = r
    note: (-)negative remains becasue gravity acts downward on falling
    object

r2 = r1' = sign(r')*C_d*r'^2 = r'
r3 = r2' = r1 - r2 = -m*sign(r)*g(r) - sign(r')*C_d*r'^2
%}
%set of linear equations with respect to r

slr=@(time,r) [r(2) ;...
    -sign(r(1))*(4*pi/3)*G*rho*r(1)-sign(r(2))*C_d*r(2)^(2)./m];
[time,r] = ode45(slr,tspan,r_0);
%%
%% Plotting ode45 solution for time span = 3 x 10^5 s
figure(1)
txt=["Postion and Velocity Of a Person Falling from Sea level to the
core";...
    "Governing equation of motion:";...
    "m*dv/dt = m*sign(r)g(r)-sign(v)*C_d*v^2";...

```

```

    "g(r) = (4π/3)*G*ρ*r";...
    "";...
    "Constants:";...
    "G = 6.674 × 10-11 m3 / kg*s2 - Gravitaional Constant";...
    "ρ = 5515 kg/m3 - average density of Earth";...
    "m= 65.8 kg , C_d = 0.25 kg/m";...
    "Inital Conditions: r = 6.371 × 106 m , v_o = 0 m/s";...
    "";...
    "Solved w/ ode45 through time span t = 3 × 105 s ~ 80 hrs"];

```

% Distance plot

```

    subplot(2, 1, 1);
    plot(time, r(:, 1));
    title(txt(1),txt(2:end),"HorizontalAlignment","center");
    xlabel('Time (s)');
    ylabel('Height above center of Earth (m)');
    grid on

```

% Velocity plot

```

    subplot(2, 1, 2);
    plot(time, r(:, 2));
    xlabel('Time (s)');
    ylabel('Velocity (m/s)');
    grid on

```

```
%%
```

```
%% plotting ode45 solution for time span = 100 s for comparison against ✓
midpoint method
```

```

tspan=linspace(0,100);
[time,r] = ode45(slr,tspan,r_0);
txt(12,1)="Solved w/ ode45 through time span t = 100 s";
figure(2)

```

% Distance plot

```

    subplot(2, 1, 1);
    plot(time, r(:, 1));
    title(txt(1),txt(2:end),"HorizontalAlignment","center");
    xlabel('Time (s)');
    ylabel('Height above center of Earth (m)');
    grid on

```

% Velocity plot

```

    subplot(2, 1, 2);
    plot(time, r(:, 2));
    xlabel('Time (s)');
    ylabel('Velocity (m/s)');

```

```

    grid on
%%
%% Solving for position and velocity functions using midpoint method
dt = 0.5; % step value
t = (0:dt:100); % dont have to do the whole time frame for midpoint method
y = zeros(length(t),2);
y(1,1) = r(1);
y(1,2) = 0;

% Solve using the midpoint method
for i = 1:length(t)-1
    y_half = y(i, :) + (dt/2) * slr(t(i), y(i, :))'; %
    y(i+1,:) = y(i, :) + dt*slr(t(i)+dt/2, y_half)';
end
%%
%% Plotting Midpoint Method solution for time span = 100 s for comparison✓
against ode45 solution
figure(3)
txt(12,1)="Solved using Midpoint Method through time span t = 100 s";
% Distance plot
subplot(2, 1, 1);
plot(t, y(:, 1));
title(txt(1),txt(2:end), "HorizontalAlignment", "center");
xlabel('Time (s)');
ylabel('Height above center of Earth (m)');
grid on

% Velocity plot
subplot(2, 1, 2);
plot(t, y(:, 2));
xlabel('Time (s)');
ylabel('Velocity (m/s)');
grid on
%%

```

Position and Velocity Of a Person Falling from Sea level to the core

Governing equation of motion:

$$m \frac{dv}{dt} = m \cdot \text{sign}(r)g(r) - \text{sign}(v) \cdot C_d v^2$$

$$g(r) = \left(\frac{4\pi}{3}\right) G \rho r$$

Constants:

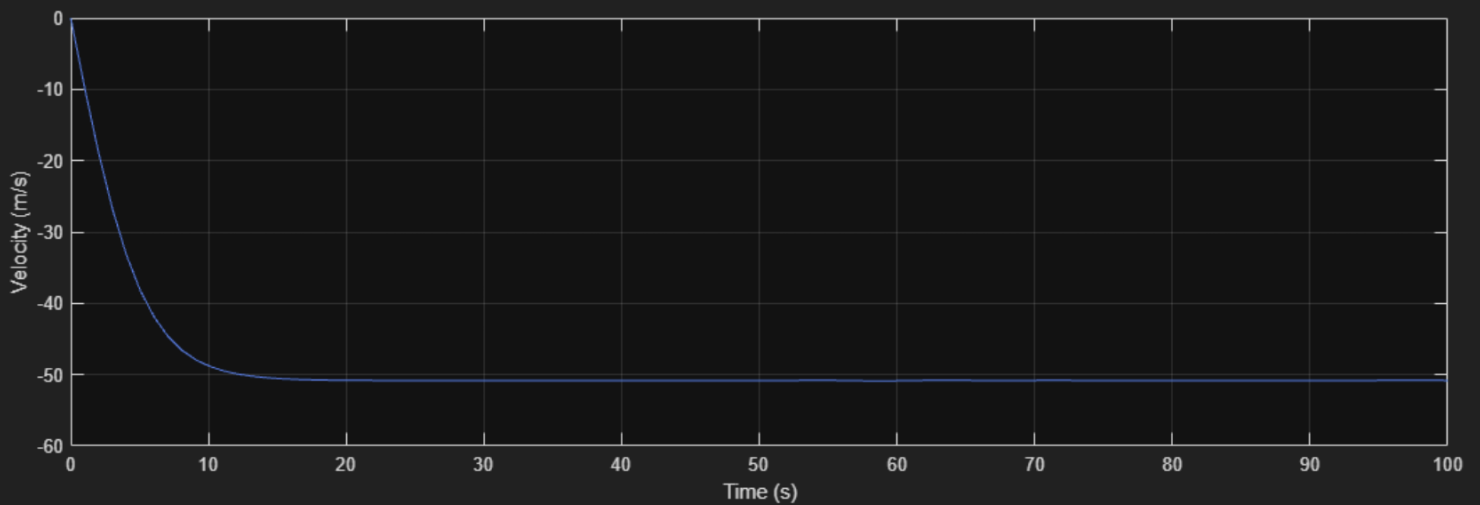
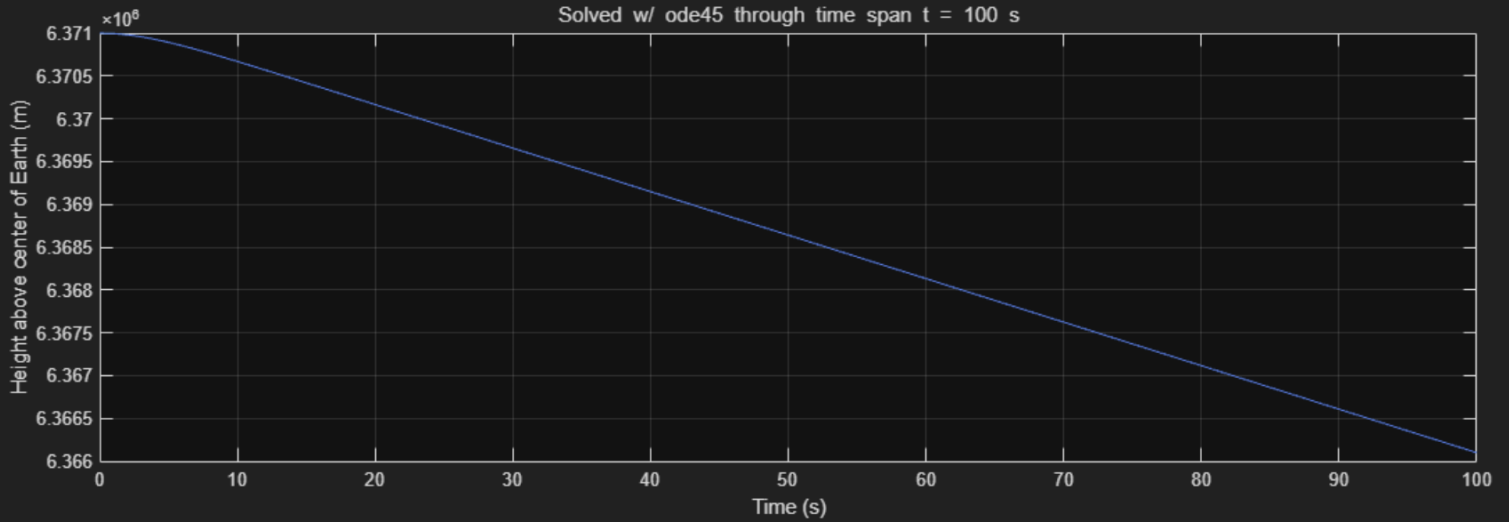
$G = 6.674 \times 10^{-11} \text{ m}^3 / \text{kg} \cdot \text{s}^2$ - Gravitational Constant

$\rho = 5515 \text{ kg/m}^3$ - average density of Earth)

$m = 65.8 \text{ kg}$, $C_d = 0.25 \text{ kg/m}$

Initial Conditions: $r = 6.371 \times 10^6 \text{ m}$, $v_0 = 0 \text{ m/s}$

Solved w/ ode45 through time span $t = 100 \text{ s}$



Position and Velocity Of a Person Falling from Sea level to the core

Governing equation of motion:

$$m \frac{dv}{dt} = m \cdot \text{sign}(r) g(r) - \text{sign}(v) \cdot C_d v^2$$

$$g(r) = \left(\frac{4\pi}{3} \right) G \rho r$$

Constants:

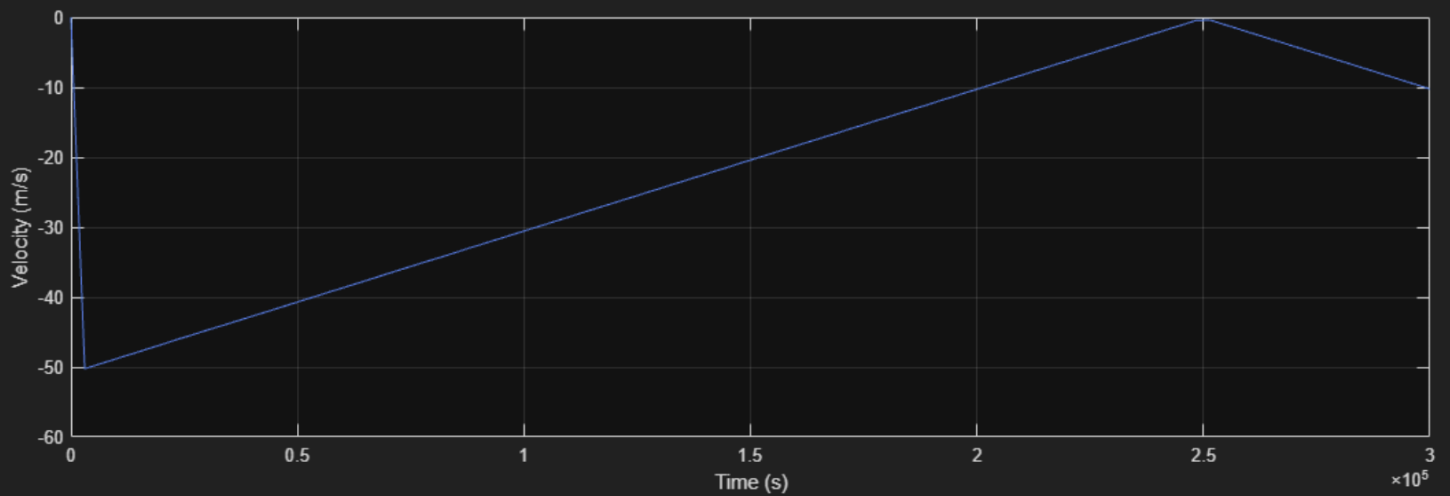
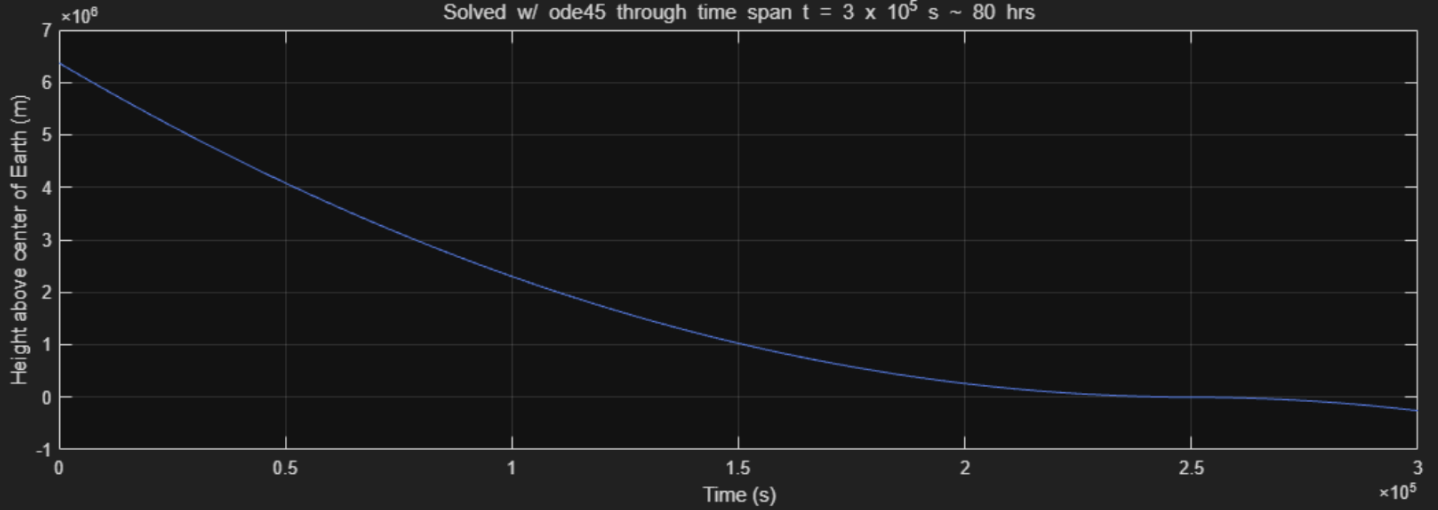
$G = 6.674 \times 10^{-11} \text{ m}^3 / \text{kg} \cdot \text{s}^2$ - Gravitational Constant

$\rho = 5515 \text{ kg/m}^3$ - average density of Earth)

$m = 65.8 \text{ kg}$, $C_d = 0.25 \text{ kg/m}$

Initial Conditions: $r = 6.371 \times 10^6 \text{ m}$, $v_0 = 0 \text{ m/s}$

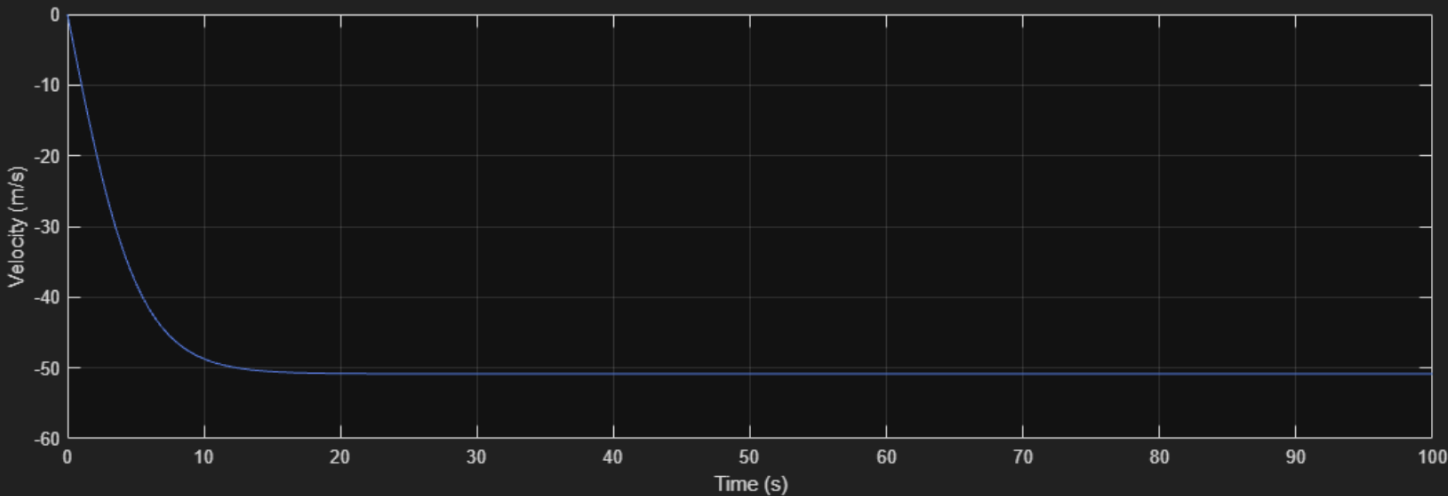
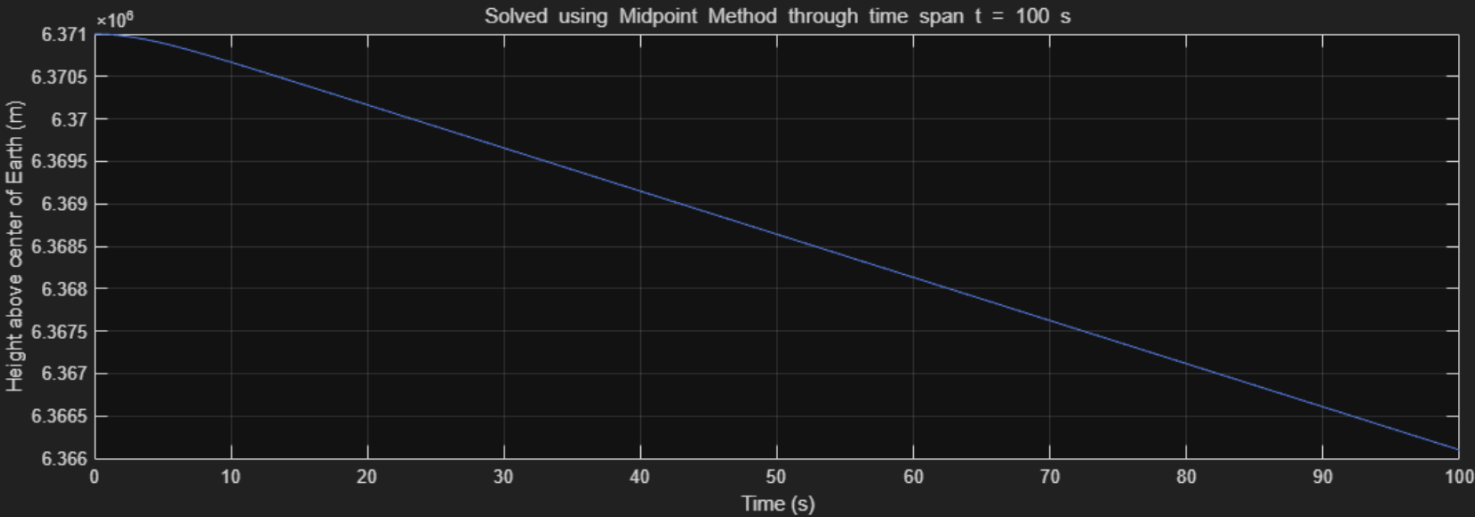
Solved w/ ode45 through time span $t = 3 \times 10^5 \text{ s} \sim 80 \text{ hrs}$



Postion and Velocity Of a Person Falling from Sea level to the core

Governing equation of motion:
 $m \cdot dv/dt = m \cdot \text{sign}(r)g(r) - \text{sign}(v) \cdot C_d^2 v$
 $g(r) = (4\pi/3) \cdot G \cdot \rho \cdot r$

Constants:
 $G = 6.674 \times 10^{-11} \text{ m}^3 / \text{kg} \cdot \text{s}^2$ - Gravitaional Constant
 $\rho = 5515 \text{ kg/m}^3$ - average density of Earth)
 $m = 65.8 \text{ kg}$, $C_d = 0.25 \text{ kg/m}$
Inital Conditions: $r = 6.371 \times 10^6 \text{ m}$, $v_o = 0 \text{ m/s}$



Problem 2 Approximating ODE

a) Analytically obtain Exact Solution

$$\text{IVP: } y'' + 7y' + 10y = 20x + 4 \quad \left| \begin{array}{l} y(0) = 2 \\ y'(0) = -10 \end{array} \right.$$

for the homogeneous solution:

$$y_h = y'' + 7y' + 10y = 0 \quad \left| \begin{array}{l} y = e^{\lambda x} \\ y' = \lambda e^{\lambda x} \\ y'' = \lambda^2 e^{\lambda x} \end{array} \right. \quad \begin{array}{l} \therefore y_h' = 10e^{\lambda x} + 7\lambda e^{\lambda x} + \lambda^2 e^{\lambda x} = 0 \\ \Rightarrow 10 + 7\lambda + \lambda^2 = 0 \\ \Rightarrow (\lambda + 2)(\lambda + 5) = 0 \\ \Rightarrow (\lambda_1, \lambda_2) = (-5, -2) \end{array}$$

$$\therefore y_h' = C_1 e^{\lambda_1 x} + C_2 e^{\lambda_2 x}$$

Since $2x + 4$ is of the form $F(t) = A_1 x + A_0$

$$\therefore y_p = K_0 + K_1 x \quad y_p' = K_1 \quad y_p'' = 0$$

Sub y_p back into Original Governing Equation

$$7K_1 + 10(K_0 + K_1 x) = 20x + 4$$

$$\Rightarrow 7K_1 + 10K_0 = 4 \quad \& \quad 10K_1 x = 20x$$

$$\Rightarrow 14 + 10K_0 = 4$$

$$\Rightarrow K_1 = 2$$

$$\Rightarrow K_0 = -1$$

$$\therefore y_p = 2x - 1$$

$$\therefore y = y_h + y_p \quad \therefore y = C_1 e^{\lambda_1 x} + C_2 e^{\lambda_2 x} + 2x - 1$$

$$y = C_1 e^{-5x} + C_2 e^{-2x} + 2x - 1$$

$$\text{Set } y(0) = 2 = C_1 + C_2 - 1 \Rightarrow C_1 = 3 - C_2$$

$$y'(0) = -10 = -5C_1 - 2C_2 + 2 = -10$$

$$\Rightarrow -5(3 - C_2) - 2C_2 + 2 = -10$$

$$\Rightarrow 3C_2 - 13 = -10 \Rightarrow C_2 = 1$$

$$\Rightarrow C_1 = 2$$

$$\therefore \boxed{y(x) = 2x + 2e^{-5x} + e^{-2x} - 1} \rightarrow \text{exact Solution}$$

Backward Euler

$$\left. \frac{dy}{dx} \right|_{x_{i+1}}^{\text{BD}} = f(x_{i+1}, y(x_{i+1})) = \frac{y(x_{i+1} + \Delta x) - y(x_i)}{\Delta x}$$

$$\Rightarrow y_{i+1} = y_i + \Delta x f(x_{i+1}, y_{i+1})$$

$$\text{for: } y_1 = y \Rightarrow y'_1 = y_2$$

$$y_2 = y' \quad y'_2 = 20x + 4 - 10y_1 - 7y_2$$

$$\Rightarrow y_{1,i+1} = y_{1,i} + \Delta x y_{2,i+1}$$

$$y_{2,i+1} = y_{2,i} + \Delta x (20x + 4 - 10y_{1,i+1} - 7y_{2,i+1})$$

$$\Rightarrow y_{1,i+1} - \Delta x y_{2,i+1} = y_{1,i}$$

$$10 y_{1,i+1} + (1 + 7\Delta x) y_{2,i+1} = y_{2,i} + \Delta x (20x + 4)$$

$$\therefore \begin{bmatrix} 1 & -\Delta x \\ 10 & (1 + 7\Delta x) \end{bmatrix} \cdot \begin{bmatrix} y_{1,i+1} \\ y_{2,i+1} \end{bmatrix} = \begin{bmatrix} y_{1,i} \\ y_{2,i} + \Delta x (20x + 4) \end{bmatrix}$$

$[A] \qquad \qquad [sol] \qquad \qquad [b]$

$$\Rightarrow [sol] = [b] \cdot [A]^{-1} \Rightarrow [sol] = A \setminus b \quad \text{matlab syntax}$$

```

clear
clc
clf
%% Approximating ODEs Problem 2
%{
Given the Initial Value Problem:


$$y'' + 7y' + 10y = 20x + 4$$


W/ initial conditions [ y(0) = 2 , y'(0) = -10]

Solve:
    * Nuemerically using both forward and backward euler methods
    * Over x-span 0 <= x <= 4
    * Vary step for each method for the given Delta_x = 0.1,0.3,0.6
Plot:
    * Plot each step size for both methods against the eaxt solution

Exact Solution(As found analytically in written work):


$$y(x) = 2x + 2e^{-5x} + e^{-2x} - 1$$

%}
%%
%% Exact Solution / Plot
y_exact=@(x) 2*x + 2*exp(-5*x) + exp(-2*x) - 1;
x=linspace(0,10);
for k=1:2
figure(k)
plot(x,y_exact(x));
title("Y-exact")
grid on
end
%%
%% Backward Euler Approximation / Plot
%

dx = [0.1,0.3,0.6,]; % step value
txt=string("step "+dx);
txt=["Exact Solution" txt];

figure(1)
hold on
% Solve using the Backward Euler Regrestion
for j = 1:length(dx)

```

```

x = (0:dx(j):6);
y=zeros(length(x),2);
y(1,1)=2;
y(1,2)=-10;
% A And b defines coeffiecients of Backward Euler recursion
A = [1,-dx(j) ;10*dx(j),(1 + 7*dx(j))];
for i = 1:length(x)-2
    b = [y(i,1); y(i,2) + dx(j)*(20*x(i+1) + 4)];
    y_solS=A\b;
    y(i+1,:)=y_solS;
end
%%
%% Plotting
plot(x, y(:,1)');
end
title("Exact Vs Backward Euler")
legend(txt)
axis([0 4 0 10])
ylabel('Y Solutions')
xlabel('X span')
hold off
%%
%% Forward Euler Approximation / Plot
figure(2)
hold on

%%
y_ip1=zeros(length(x),2);
y = zeros(length(x),2);
for j = 1:length(dx)
    x=(0:dx(j):100);
    y = zeros(length(x),2);
    y(1,1) = 2;
    y(1,2) = -10;
    for i = 1:length(x)-1
        %solves for y(x+h) sly returns [y(x+h),y'(x+h)]
        y(i+1,:) = y(i,:) + dx(j).*sly(x(i), y(i,:))';
    end

%% Plotting
plot(x, y(:, 1));
end
title("Exact Vs Forward Euler")
legend(txt)

```

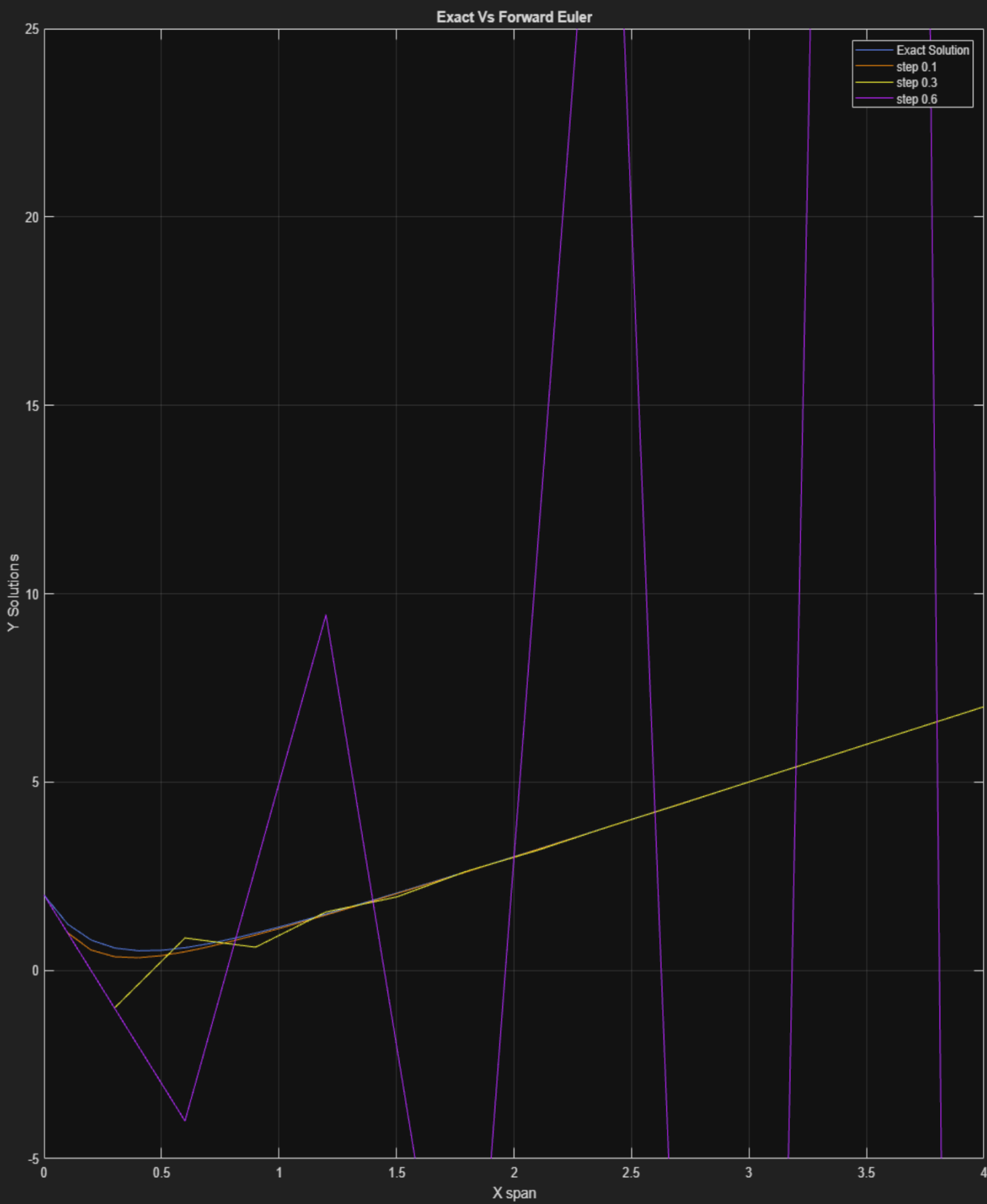
```
axis([0 4 -5 25])
ylabel('Y Solutions')
xlabel('X span')
grid on

%%
%% Set up Sys. 1st Order Linear y for used in solving Forward and Backward✓
Eulers

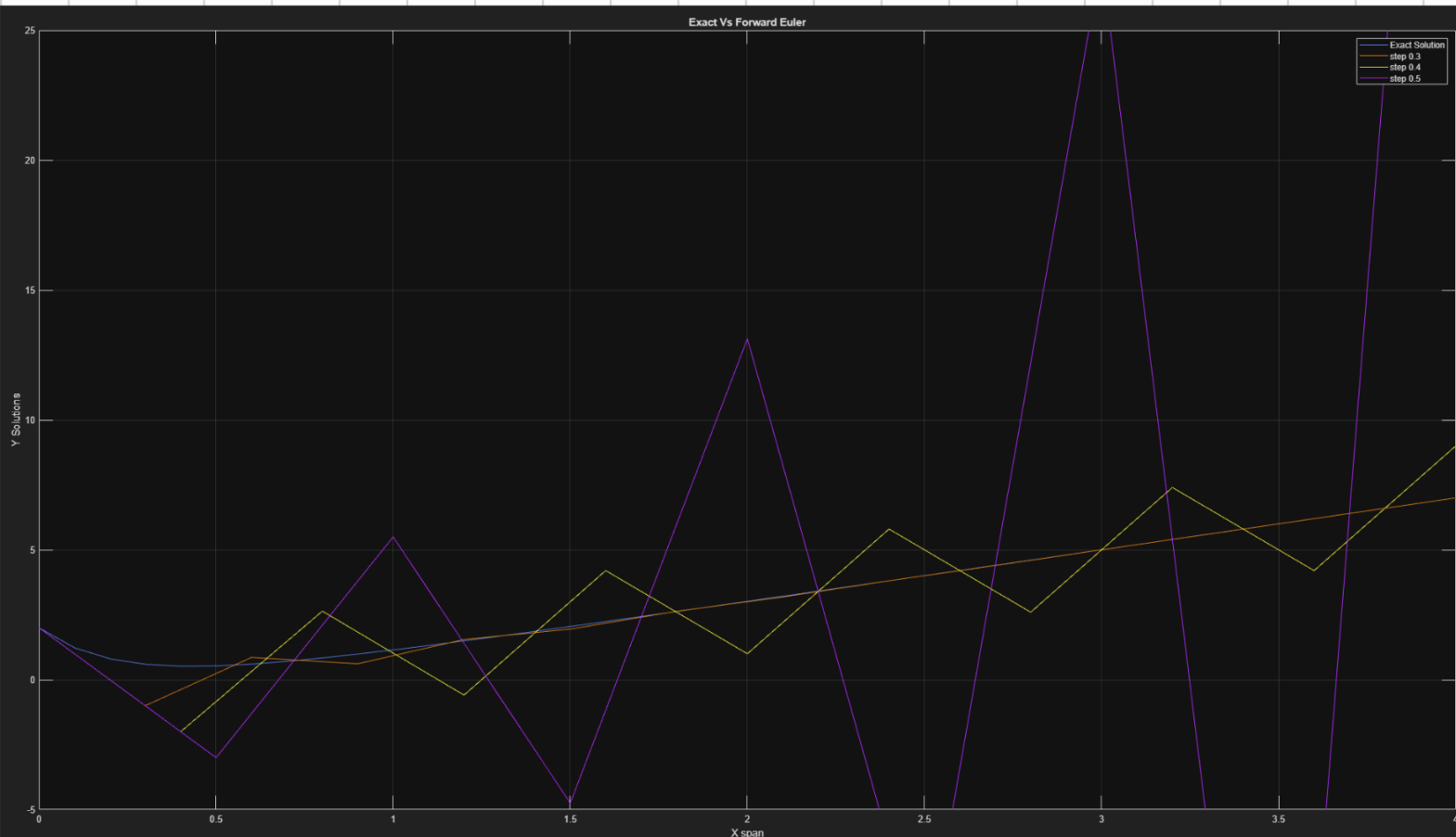
function y_p = sly(x,y) % y_p = y'
    y_p = zeros(2, 1); % Column vector'
    y_p(1) = y(2); % y_p(1) = y1' = y2
    y_p(2) = 20*x+4-10*y(1)-7*y(2); % y_p(2) = y2' = y''
end
```

Exact Vs Backward Euler





C) The first signs of instability for the Forward Euler method can be seen @ a step size of 0.3 which is most notably seen during $x_{span} < 1.5$ where solutions flip back and forth between under predicting and over predicting before it stabilizes and fits with the exact.



It becomes Sully unstable at step values < 0.4 as shown in the figure above. where we can see that at a step of 0.4 it still follows the trend of the exact solution. but as we get to a step of 0.5 it completely becomes unstable and no trend can be discerned.

d) The Backward Euler Method does Not become unstable. The Error is Not affected as you increase step size
 $\therefore$ Backward Euler is unconditionally stable


```
%% As3-ME373 Q3
```

```
clear
```

```
clc
```

```
clf
```

```
%% BVP - Shooting Method
```

```
%{
```

```
The steady- state height of water in a 1-dimensional, unconfined ground  
water aquifer can be modeled with the following nonlinear ODE:
```

$$((K*h)dh/dx)d/dx + N = 0$$

```
Where:
```

```
x = distance (m)
```

```
h = height of water table (m)
```

```
K = Hydraulic-conductivity (m/d)
```

```
N = Infiltration rate (m/d)
```

```
given:
```

```
0 ≤ x ≤ 1000 m
```

```
h(x = 0) = 10 m
```

```
h(x = 1000) = 5 m
```

```
K = 1 m/d
```

```
N = 0.0001 m/d
```

```
Solve:
```

```
For the water height using the shooting method(Use ode45 to integrate)  
plot X vs h.
```

```
Hint: use product rule to outer derivate.
```

```
%}
```

```
%% Initial Setup
```

```
%{
```

$$((K*h)dh/dx)d/dx + N = 0$$

```
Product rule -> f'g + fg'
```

$$\rightarrow d(K*h)/dx * dh/dx + K*h *d(dh/dx)dx + N = 0$$

$$\rightarrow K(h')^2 + Kh*h'' + N = 0$$

$$\rightarrow h'' + ((h')^2)/h + N/(Kh) = 0$$

```
Therefor set up system to linearize Non-linear ODE
```

```
[y1,y2]=[h,h']
```

```

[y1',y2']=[y2, -(y2^2)/y1 - N/(K*y1)]
%}
format long

x =[0,1000];
h =[10,0];
% Guess range of h'(0) needs to be realtively small because we have a
% height decay of 5m over 1000m distance
guess= [0.0009999,0.0013125];
K = 1;
N = 0.0001;

% system linearized
slh_Shooting=@(x,h) [h(2);-h(2)^2/(h(1))-N/(K*h(1))];

% Target
h_target = 5;
AbsTol=exp(-11);
hold on
% Solves system using ode45| Iinital Conditions h(0)= 10 , h'(0)= guess
% to find h(x=1000) for each guess and stores it for interoplation to
% find h'(0) actual that fits height target.
for i = 1:length(guess)
    h(2)=guess(i);
    sol_mat(:,i)=ode45(slh_Shooting,x,h,odeset('AbsTol',AbsTol));
    sol(i) = sol_mat(i).y(1,end);
    plot(sol_mat(i).x(:),sol_mat(i).y(1,:), '--');
    guess_str(i,:)=num2str(guess(i));
    if i==2
        % Shooting Methond Interpolation of guesses and solutions
        m = (guess(1)-guess(2))/(sol(1)-sol(2));
        guess(1,3) = guess(2) + m*(h_target-sol(2));

        h(2)=guess(i+1)
        sol_mat(:,i+1)=ode45(slh_Shooting,x,h,odeset('AbsTol',AbsTol));
        sol(i+1) = sol_mat(i+1).y(1,end);
        plot(sol_mat(i+1).x(:),sol_mat(i+1).y(1,:));
        guess_str(i+1,:)=num2str(h(2));
    end
end
grid on;
txt=["Guess 1 h'(0)= ", "Guess 2 h'(0)= ", "Guess 3 h'(0)= "];
txtStr=strcat(txt,guess_str)

```

```
title('Groundwater Travel Distance vs Aquifer Water Height ');  
xlabel('Groundwater Travel Distance x-(m) ');  
ylabel(' Aquifer Water Height h-(m) ');  
legend(txtStr(1),txtStr(2),txtStr(3));
```

